

1 Описание модели анализа аудиозаписей на предмет выявления признаков-маркеров дислексии

Реализация модели выполнена на языке Python, основной модуль – файл `dysmarkcccls_clean.py` в каталоге `model` репозитория проекта <https://github.com/dysata/dyslexia-child/>.

Входом модели является аудиозапись чтения текста (или названия объекта на изображении) и соответствующий текст. Выходом модели является последовательность троек: <ожидаемая фонема, распознанная фонема, код признака-маркера>. Ожидаемая фонема – фонема, которая ожидается в соответствующем месте аудиозаписи на основании сопоставления с текстом, предлагавшемся для произнесения испытуемым. Признак-маркер – это значимая в диагностике дислексии специфическая ошибка произношения такая как замены фонем, заикание и т.д.. См. список далее.

В целом задача ставится как задача сопоставления ожидавшегося (на основе предъявленного к прочтению текста) потока фонем/пауз и распознанного потока фонем/пауз с выявлением ошибок произношения. Здесь требуется решить три ключевых подзадачи:

- Распознать поток фонем в аудиозаписи.
- С учетом возможных ошибок в произношении текста испытуемым выровнять последовательности относительно друг друга. Например, имеем последовательность к-р-о-т, а испытуемый произносит к-к-к-л-о-т. Необходимо корректно сопоставить “к” в ожидавшейся последовательности подпоследовательности “к-к-к” в распознанной последовательности, “р”-“л”, и т.д., так как нарушение совмещения приводит к тому, что правильно произнесенные фонемы будут промаркированы как ошибочные).
- Классифицировать расхождения по признакам-маркерам ошибок.

В связи особенностями этих задач, оказалось целесообразным использовать гибридный подход к построению архитектуры модели анализа аудиозаписи: он сочетает несколько методов решения задач, в т.ч. методы искусственного интеллекта: модели машинного обучения – нейросеть и модель градиентного бустинга на деревьях решений, а также модель основанную на продукционных правилах с упорядочиванием:

1. На первом этапе анализа осуществляется транскрибация аудиозаписи, для чего используется нейросеть `dysata/Wav2Vec2-Ru-Child` (размещена на Hugging Face <https://huggingface.co/dysata/Wav2Vec2-Ru-Child>). Нейросеть получена в проекте на основе нейросети `wav2vec2[1]` в результате дообучения на подготовленном в рамках проекта датасете, размещенном на Hugging Face:

- <https://huggingface.co/datasets/dysata/childread001> – чтение детьми текстов.

В результате работы модели получается:

- транскрибация – последовательность фонем, распознанных нейросетью,
- эмиссионная матрица – матрица E , где каждый элемент $E(t,v)$ – это оценка нейросетью вероятности того, что в сегменте t аудиозаписи произносилась фонема v . Длина сегмента составляет около 20 миллисекунд.

При этом может быть так, что читающий делит фонему или паузу в течение нескольких сегментов, и это отслеживается в эмиссионной матрице, а в транскрибации нейросеть повторы букв и затянувшиеся паузы устраняет.

2. На втором этапе – Forced Alignment – реализуется определение фактических сегментов звучания фонем/пауз. Т.е., если эмиссионная матрица показывает, что одна фонема слышится в течение нескольких сегментов, на втором этапе анализа будет построен объединенный сегмент, чтобы информация о временной привязке фонем/пауз получилась в явном виде.

Такая привязка выполняется для двух последовательностей:

- эталона – исходного текста, который предъявлялся к прочтению испытуемым,
- транскрибации, полученной на первом этапе.

Реализация второго этапа основана на алгоритме Витерби для поиска оптимального пути сопоставления последовательности токенов текста с кадрами акустической эмиссионной матрицы модели (см. файл `model/w2vtransf.py` в репозитории). На выходе для каждого из входных текстов (эталона и транскрибации) получаются последовательности фонем/пауз, маркированных временем начала и конца сегмента звучания фонемы/паузы. Таким образом получается первичное выравнивание двух потоков фонем/пауз относительно шкалы времени. Однако тут не учитывается такая проблема: если ребенок заикается и произносит “к-к-к-о-т”, то для транскрибации будет получено 3 сегмента с фонемой “к”, а для исходного текста эти сегменты записи будут объединены в один, как если бы “к” звучала долго. Следующая проблема: “лишние” фонемы/паузы в транскрибации – когда испытуемый делает большие задержки или произносит звуки, не предусмотренные текстом. Выравнивание исходного текста с аудиозаписью в таком случае растянет сегменты правильных фонем, а для каждого неверного звука/паузы в выравнивании транскрибации будет сопоставлен сегмент аудио. Следующая проблема – с пропавшими, не воспроизведенными звуками/паузами. Для решения этих проблем выполняется третий этап.

3. Две последовательности сегментов идут на вход третьему этапу – Remapping. Здесь осуществляется поправка сегментного выравнивания на основе деления текста на слова

(что позволяет правильно сегментировать участки “слово” “пауза”), и на основе сопоставления подпоследовательностей фонем так, чтобы в первую очередь сопоставить правильно произнесенные подпоследовательности, а далее уже на этой основе установить соответствие между сегментами с наблюдаемыми расхождениями. Так в конечном итоге находятся соответствующие друг другу участки аудио, где распознанные фонемы совпадают с ожиданием и участки, где проявляются расхождения.

Алгоритм осуществляет итеративное сопоставление слов справа налево, выбирая для каждого слова эталона наиболее позднее допустимое совпадение в гипотезе (точное или по префиксу), не нарушающее хронологический порядок предыдущих слов. На основе найденных соответствий алгоритм заменяет временные метки сегментов эталона на метки из гипотезы, тем самым принудительно привязывая эталонную текстовую последовательность к реальным акустическим границам, включая заикания и удлинения. Также выполняется коррекция позиций пауз. Выходом алгоритма является последовательность пар (r_i, h_i) , где h_i – распознанная фонема или пауза, r_i – сопоставленная этой фонеме фонема из эталонного текста, либо признак ошибки (он выставляется для выявленных расхождений типа повтор или лишняя пауза).

4. На четвертый этап поступает эта последовательность пар (r_i, h_i) . Реализуется выявление признаков-маркеров, которое осуществляется двумя способами:

4.1 на основе модели основанной на правилах (rule based model, продукционная модель с упорядоченными правилами).

Имеется упорядоченный набор функций-детекторов $F = \{f_1, f_2, \dots, f_k\}$. Алгоритм реализует последовательное применение функций к входной последовательности, добавляя к каждой паре маркер ошибки m_i , в результате получаем тройку (r_i, h_i, m_i) . Если маркер был выставлен, другие правила к этой паре не применяются. Обработка последовательности на шаге k заключается в применении f_k к каждой паре i в последовательности: выполняется анализ контекстного окна $W_i = \{(r_{i-w}, h_{i-w}), \dots, (r_{i+w}, h_{i+w})\}$. Если конфигурация символов в окне удовлетворяет предикату $P_k(W_i)$, то происходит присвоение значения маркера $m_i \leftarrow code_k$, если ранее маркер не был проставлен предыдущей функцией-детектором. Продукционные правила в порядке их применения приведены в Таблице 1.

Таблица 1 — Продукционные правила

Код признака-маркера	Название ошибки	Предикат
К	Заикание	если в гипотезе идет серия из двух и более одинаковых фонем, и хотя бы одна из них совпадает с

		эталон, то все несовпадающие позиции в этой серии (кроме пауз и пропусков гласных) помечаются маркером К
7	Замена по звонкости/глухости	Выявление замен по звонкости/глухости (например, 'д'→'т', 'б'→'п'). Алгоритм исключает из ошибок случаи нормального фонетического оглушения: когда звонкий согласный стоит после гласной и перед глухим, либо в конце слова. Если контекст указывает на нормальное оглушение, маркер сбрасывается в '0', иначе ставится маркер '7'.
1	испытываемый путает Ж-Х	Выявление специфической парной замены фонем «ж» ↔ «х». Если в гипотезе обнаружен один из этих звуков, а в эталоне на той же позиции стоит парный ему звук, сегмент еще не помечен другими правилами, ему присваивается маркер 1
2	Н-П, И-Н	Выявление специфических парных замен: «н» ↔ «п» и «и» ↔ «н». Если в гипотезе обнаружен один из этих звуков, а в эталоне на той же позиции стоит соответствующий парный звук, и сегмент еще не помечен, ему присваивается маркер 2.
3	В-З	Выявление специфической парной замены фонем «в» ↔ «з». Если в гипотезе обнаружен один из этих звуков, а в эталоне на той же позиции стоит парный ему звук, и сегмент еще не помечен другими правилами, ему присваивается маркер 3
4	Т-Г	Выявление специфической парной замены фонем «т» ↔ «г». Если в гипотезе обнаружен один из этих звуков, а в эталоне на той же позиции стоит парный ему звук, и сегмент еще не помечен другими правилами, ему присваивается маркер 4.
5	Р-Ь-Б	Выявление специфических парных замен в группе «р», «ь» (мягкий знак), «б». Алгоритм проверяет циклические соответствия: «р»↔«ь», «ь»↔«б», «б»↔«р». Если в гипотезе обнаружен один из этих звуков, а в эталоне на той же позиции стоит соответствующий парный звук из списка, и сегмент еще не помечен, ему присваивается маркер 5.
6	О-Ю	Выявление специфической парной замены гласных «о» ↔ «ю». Если в гипотезе обнаружен один из этих звуков, а в эталоне на той же позиции стоит парный ему звук, и сегмент еще не помечен другими правилами, ему присваивается маркер 6.
8	Х-К	Выявление специфической парной замены заднеязычных согласных «х» ↔ «к». Если в гипотезе обнаружен один из этих звуков, а в эталоне на той же

		позиции стоит парный ему звук, и сегмент еще не помечен другими правилами, ему присваивается маркер 8.
Н	Пропуск с удвоением	Если в гипотезе стоит пауза (), а в эталоне на этом месте и на предыдущем стоит одинаковая согласная, которая в предыдущей позиции совпала корректно. Это фиксирует некорректный разрыв между повторяющимися звуками.
9	Пропуск согласной на стечении согласных	Если текущий звук в эталоне – согласная, следующий в гипотезе – тоже согласная, но они не совпадают друг с другом, при этом одна из этих фонем пропущена в распознанной последовательности. Это указывает на выпадение или искажение одной из согласных в кластере (стечении).
Ј	Повтор через паузу / лишний префикс	Выявление повторений звука через паузу (заикание типа «к... к») и удлинения префикса слова. Повтор с паузой: Если звук в гипотезе не совпадает с эталоном, за ним сразу следует пауза (), а после паузы этот же звук повторяется. Лишний префикс: Если в эталоне ожидается пауза (граница слова), а в гипотезе произнесен звук (согласный или гласный), который не является простым продолжением предыдущего звука (не удлинение) и не следует сразу за другой паузой в эталоне.
А	Удлинение гласной с пропуском согласной	Выявление удлинения гласной с пропуском согласной. Срабатывает, если в эталоне ожидается согласная, а в гипотезе на этой позиции произнесена гласная, которая идентична предыдущей гласной в гипотезе (ребенок «застрял» на гласной вместо перехода к согласной).
В	Пропуск гласной между согласными	Выявление пропуска гласной между двумя согласными. Срабатывает, если в эталоне на текущей позиции стоит гласная, окруженная согласными (предыдущая в эталоне — согласная), а в гипотезе на этом месте произнесена согласная. Это соответствует паттерну искажения «согласная-гласная-согласная» → «согласная-согласная-согласная».
Л	Удлинение согласной с пропуском гласной	Выявление удлинения согласной с пропуском гласной. Срабатывает, если в эталоне ожидается гласная, а в гипотезе на этой позиции произнесена согласная, которая идентична предыдущей согласной в гипотезе. Это соответствует ситуации, когда ребенок «застревает» на предыдущем согласном звуке вместо того, чтобы произнести следующую гласную (паттерн: эталон ...СГ → гипотеза ...СС).

D	Перестановка гласных	Выявление перестановки (транспозиции) двух гласных звуков. Алгоритм ищет пару позиций i и j , где на месте ожидаемой гласной X произнесена гласная Y , а на месте ожидаемой Y (в пределах последующего контекста) произнесена X . Если такая взаимная замена обнаружена и сегменты еще не помечены, обеим позициям присваивается маркер D.
G	Смягчение окончания	Выявление смягчения гласной в конце слова перед границей. Срабатывает, если в эталоне стоит твердая гласная (а, о, у), а в гипотезе на её месте стоит соответствующая мягкая пара (я, ё, ю), и сразу за этим сегментом в эталоне следует граница слова () или конец последовательности.
F	Лишний звук в конце слова	Выявление лишнего звука в конце слова. Срабатывает, если в эталоне на текущей позиции стоит граница слова (), а в гипотезе произнесен звук (согласный или гласный). При этом проверяется, что предыдущий символ в эталоне не был границей (чтобы не маркировать начало фразы) и что добавленный звук не является простым продолжением/удлинением предыдущего звука (слиянием). Если условия выполнены, сегмент помечается маркером F.
C	Произвольная замена (остаточная) или пропуск	Произвольная замена (остаточное правило). Срабатывает для любого несовпадения эталона и гипотезы, которое не было помечено предыдущими правилами. Исключаются границы слов () и случаи простого удлинения звука (когда текущий звук в гипотезе идентичен предыдущему), так как они считаются частью другого дефекта или нормальным варьированием.
I	Ошибка произнесения R	Применяется классификатор на основе CatBoost, см. п. 4.2 ниже
E	Перестановка согласных	Выявление перестановки (транспозиции) двух согласных звуков. Алгоритм ищет пару позиций i и j , где на месте ожидаемой согласной X произнесена согласная Y , а на месте ожидаемой Y (в пределах последующего контекста) произнесена X . Если такая взаимная замена обнаружена и оба сегмента еще не помечены, им присваивается маркер E.
0	нет ошибки	выставляется, если не были выставлены другие маркеры

4.2 на основе модели градиентного бустинга на деревьях решений CatBoost [3], обученной на подготовленном в рамках проекта датасете с подборкой хороших и плохих

произношений “p” (<https://huggingface.co/datasets/dysata/rwords>). Обученная модель размещена в репозитории проекта в файле model/catboost_r_model.cbm.

Это частное решение для выявления характерного маркера – нарушение произношения “p”. В его основе лежит извлечение информации с последнего скрытого слоя нейросети wav2vec2, где информация о звуке каждого 20мс-сегмента аудиозаписи и его контексте представлена вектором в пространстве 1024 признаков. Различные нарушения произношения “p” формируют в этом пространстве выразительные кластеры. Таким образом, интерес представляет классификация нарушений на основании отнесения распознанной фонемы к тому или иному кластеру. Для классификации векторов в этом пространстве собрана выборка хороших и плохих произношений “p” и обучена модель CatBoost. На этой основе получается следующий гибридный алгоритм распознавания и классификации качества произношения “p”:

1. Как было объяснено выше, на этапах 1-3 с помощью wav2vec2, forced alignment и remap получается последовательность сегментов аудио, соответствующих распознанным фонемам (это сегменты различной длины – в зависимости от того, сколько в аудиозаписи длилась да или иная фонема/пауза), и сопоставленные этим сегментам фонемы эталонного (предлагавшегося к прочтению) текста.
2. Для каждого сегмента последовательности из п. 1, которому соответствует буква “p” из эталонного текста, определяется последовательность 20мс-сегментов аудиозаписи и выбираются их векторные представления из последнего скрытого слоя wav2vec. Вычисляется вектор представления всей длящейся фонемы как среднее этих векторов.
3. Далее применяется модель CatBoost, обученная различать хорошие и плохие “p” для классификации этого осредняющего вектора.

Таким образом, эффективно используются два инструмента для соответствующих подзадач: Wav2Vec2 используется для перевода представления звука в богатом пространстве, где похожие звуки кластеризуются рядом, даже если они произнесены разными людьми, а CatBoost решает задачу классификации в этом пространстве.

2 Порядок развертывания и использования

2.1 Развертывание программного инструментария

Развертывание разработанной модели для использования заключается в создании виртуального программного окружения Python и установке зависимостей после чего модель, реализованную в виде Python-модулей можно применять в режиме запуска модулей как программ, либо включения их в прикладные Python-программы в качестве библиотеки.

В каталоге с репозиторием проекта следует выполнить команды (тестирование выполнялось с Python версии 3.12.3):

```
sudo apt update && sudo apt upgrade -y
sudo apt install python3 python3-venv python3-pip -y
cd model
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

2.2 Порядок использования моделей машинного обучения в коде: нейросети и на деревьях решений

Порядок применения модели dysata/Wav2Vec2-Ru-Child (размещена на <https://huggingface.co/dysata/Wav2Vec2-Ru-Child>) в коде на Python показан в Python-модуле `model/dysmarkcccls_clean.py` (путь относительно репозитория проекта):

1. Требуется подключить модули для работы с архитектурой `wav2vec2`:

```
from transformers import Wav2Vec2Processor, Wav2Vec2ForCTC
```

2. Задать параметры и загрузить модель:

Параметры:

```
MODEL_NAME_PROCESSOR = "dysata/Wav2Vec2-Ru-Child"
```

```
MODEL_NAME_MODEL = "dysata/Wav2Vec2-Ru-Child"
```

```
SAMPLING_RATE = 16000
```

Создание кэша:

```
_model_cache = {
    "processor": None,
    "model": None
}
```

Загрузка модели:

```
def _load_model_cached() -> Tuple[Wav2Vec2Processor, Wav2Vec2ForCTC]:
    if _model_cache["processor"] is None or _model_cache["model"] is None:
        sys.stderr.write("Loading model (first use)...\n")
        processor = Wav2Vec2Processor.from_pretrained(MODEL_NAME_PROCESSOR)
        model = Wav2Vec2ForCTC.from_pretrained(MODEL_NAME_MODEL)
        model.eval()
        _model_cache["processor"] = processor
        _model_cache["model"] = model
    sys.stderr.write("Model loaded.\n")
```



```

    return _model_cache["processor"], _model_cache["model"]
processor, model = _load_model_cached() # процессор и модель загружены
При необходимости можно применить очистку кэша:
def clear_model_cache():
    _model_cache["processor"] = None
    _model_cache["model"] = None
3. Получить звук из аудиофайла:
audio, sr = librosa.load(audio_path, sr=SAMPLING_RATE)
sounds_in_batch = [audio]
4. Получение эмиссионной матрицы, транскрибации, последнего скрытого слоя:
def _get_emission_and_transcription(
    model: Wav2Vec2ForCTC,
    processor: Wav2Vec2Processor,
    audio: List[Any]
) -> Tuple[List[Any], str, torch.Tensor]:
    processed = processor(
        audio,
        sampling_rate=SAMPLING_RATE,
        return_tensors="pt",
        padding="longest"
    )
    with torch.no_grad():
        outputs = model(processed.input_values, attention_mask=processed.attention_mask,
        output_hidden_states=True, return_dict=True)
        logits = outputs.logits
        last_hidden_state = outputs.hidden_states[-1].cpu()
        feat_extract_output_lengths = model._get_feat_extract_output_lengths(
            processed.attention_mask.sum(dim=1)
        ).numpy()
        emission_matrices = []
        for sample_idx in range(feat_extract_output_lengths.shape[0]):
            specgram_len = feat_extract_output_lengths[sample_idx]
            new_emission_matrix = torch.log_softmax(
                logits[sample_idx, 0:specgram_len],
                dim=-1

```

```

).numpy()
    emission_matrices.append(new_emission_matrix)
    predicted_ids = torch.argmax(logits, dim=-1)
    transcription = processor.decode(predicted_ids[0])
    return emission_matrices, transcription, last_hidden_state

```

Порядок применения модели CatBoost model/catboost_r_model.cbm (файл в репозитории проекта) в коде на Python показан в Python-модуле model/marks.py (путь относительно репозитория проекта):

1. Подключение модулей

```
from catboost import CatBoostClassifier
```

2. Загрузка модели

```

model_path = os.path.join(os.path.dirname(__file__), 'catboost_r_model.cbm')
if not os.path.exists(model_path):
    print(f"markR: модель не найдена: {model_path}")
    return segments2
cb_model = CatBoostClassifier()
cb_model.load_model(model_path)

```

3. Классификация осредненного вектора представления фонемы из последнего скрытого слоя wav2vec2

```

hs = hidden_states[0, frame_idx:end_frame].numpy() # получение массива векторов по всем
20-мс сегментам, в течение которых длится произношение фонемы
hs_mean = np.mean(hs, axis=0).reshape(1, -1) # осреднение
pred = cb_model.predict(hs_mean) # обращение к классификатору
int(pred[0]) == 0 означает, что произношение “p” классифицируется как плохое

```

3 Проверка работы

3.1 Режим программы, командный интерфейс

В режиме использования модуля `dysmarkcccls_clean.py` как программы:

```
python dysmarkcccls_clean.py <путь к файлу с текстом> <путь к файлу с аудио>
```

Примеры текстовых и аудиофайлов можно найти в каталоге `test_data`

Пример вывода:

```
{
  "triples": [
    ["в", "в", "0"],
    ["н", "в", "7"],
    ["а", "а", "0"],
    ["ш", "ш", "0"]
  ],
  "quality": "good"
}
```

3.2 Режим библиотеки

Опробовать применение библиотеки можно с помощью демонстрационных Python-ноутбуков в подкаталоге `demo` проекта:

- `test_quality_checks.ipynb` - демонстрирует анализ годности аудиозаписей для исследования,
- `test_marking.ipynb` – демонстрирует анализ аудиозаписей на выявление признаков-маркеров,
- `benchmark_performance.ipynb` – исследует время анализа аудиозаписи и потребление памяти.

Для работы с ноутбуками можно использовать то же окружение, что было развернуто в каталоге `model`, но для чистоты можно развернуть отдельное программное окружение в каталоге `demo/` Для настройки окружения в каталоге с репозиторием кода выполняются команды:

```
cd demo
python3 -m venv jupyter_env
source jupyter_env/bin/activate
pip install -r requirements.txt
```

Далее нужно запустить сервис для работы с ноутбуком:

```
jupyter notebook <имя Python-ноутбука>.ipynb
```

Как правило, в результате запускается автоматически браузер и ноутбук становится доступным для работы в открытом окне браузера. Если этого не произошло, обратите внимание на то, что команда распечатывает в терминале ссылку вида:

```
http://localhost:8888/tree?token=<строка токена>
```

Её следует открыть в браузере и далее выбрать в браузере один из Python-ноутбуков: файлы с расширением `interp.ipynb`.

После работы с Python-ноутбуками для закрытия сервиса нажмите комбинацию `ctrl+c` в терминале, где был запущен сервис. Для деактивации виртуальной среды выполните команду

```
deactivate
```

Если виртуальная среда больше не требуется, удалите каталог с ее настройками:

```
rm -Rf jupyter_env
```

Литература

1. Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. 2020. Wav2vec 2.0: a framework for self-supervised learning of speech representations. In Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 1044, 12449–12460.
2. Корнев А. Н. Нарушения чтения и письма у детей: Учебно-методическое пособие. – СПб.: ИД “МиМ”, 1997. – 286 с.
3. Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. 2018. CatBoost: unbiased boosting with categorical features. In Proceedings of the 32nd International Conference on Neural Information Processing Systems (NIPS'18). Curran Associates Inc., Red Hook, NY, USA, 6639–6649.